Day 68: Stacked and Bidirectional RNNs, From Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 10, Day 6 | Complete study guide

Watch: Bidirectional RNN | Deep Learning Tutorial 38 (Tensorflow, Keras & Python) (codebasics) | Deep RNNs | Stacked RNNs | Stacked LSTMs | Stacked GRUs (CampusX)

Script: `day68_stacked_bidirectional.py` · every number in this guide is copied by program from `day68_run_output.txt`, the real output of that script.

Day 67 found that the recurrent cell barely matters for tagging, and that every tagger looks only a few words back. What none of them could do was look *forward*. Today's two standard extensions, built on Day 67's LSTM tagger: a **bidirectional** tagger, which adds a second LSTM reading right to left, raised test accuracy from 0.8377 to 0.8577, the biggest gain in Weeks 9–10. A **second layer** did not help: 0.8326 left to right and 0.8557 bidirectional.

0. Where today fits

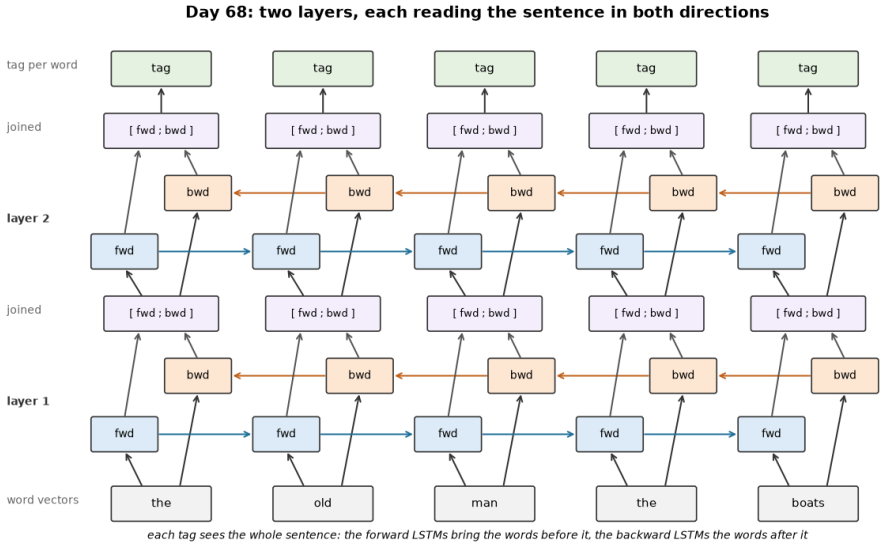
DEFINITION — start here, no prior knowledge assumed

A **bidirectional** RNN runs two recurrent networks over the same sequence, one left to right and one right to left, and joins their states at every position. A **stacked** (deep) RNN feeds the states of one recurrent layer into a second recurrent layer as its input sequence.

REAL-WORLD SCENARIO

"The old man the boats." Reading left to right, "man" looks like a noun, until "the boats" reveals it is a verb (the old people crew the boats). A careful reader looks ahead before deciding. A bidirectional tagger does the same; a stacked one re-reads its own first impressions.

SYSTEM DIAGRAM



A 2-layer bidirectional tagger. In each layer the forward states pass right and the backward states pass left; the two are joined at every word and handed up. Every tag sees the whole sentence.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Four taggers: one or two layers, one or two directions

DEFINITION — start here, no prior knowledge assumed

One **DeepTagger** class with two settings, `layers` and `bidirectional`. Every LSTM inside it is Day 63–65's code, unchanged; each layer and each direction has its own weights.

REAL-WORLD SCENARIO

Adding a second proofreader who reads each page from the bottom up (bidirectional), or a second round of proofreading that works from the first round's notes (stacked).

MATHEMATICS

$\rightarrow H = LSTM_f(X), \quad \leftarrow H = R(LSTM_b(R(X))), \quad H = [\rightarrow H ; \leftarrow H]$ *(R = reverse each sentence's real words)*

stacked: $X^{(2)} = H^{(1)}$; *tags from the top layer's H*

LSTM weights per direction: $4(D_{in} + H + 1)H$, *with* $D_{in} = 32$ *(words) or 64 (a bidirectional layer below)*

SPECIFICATION

Shared	Day 67's LSTM tagger: embedding 32, hidden 32 per direction, lr 4.0, forget bias 1.0 (Day 67's validation choice), 25 epochs, batch 32, clip 1.0.
Reproduction	The 1-layer left-to-right tagger's first LSTM uses exactly Day 67's seed, so it must reproduce Day 67's LSTM results.

Why this test

Bigger models have more weights; the table shows how many, so later comparisons can account for it.

IMPLEMENTATION

```
for l in range(layers):
    in_dim = embed_dim if l == 0 else width
    for j, d in enumerate(self.directions):
        W, b = init_lstm(in_dim, hidden_dim, seed + 1000 * (2 * l + j), forget_bias)
        setattr(self, f"W{l}{d}", W); setattr(self, f"b{l}{d}", b)
        self.params += [f"W{l}{d}", f"b{l}{d}"]
```

ACTUAL OUTPUT

```
tagger                      LSTMs  LSTM weights  output layer    total
1 layer, left to right          1          8320           561    75505
2 layers, left to right         2         16640           561    83825
1 layer, bidirectional          2         16640          1105    84369
2 layers, bidirectional         4         41472          1105   109201
Actual output from this lesson's run.
```

What it means

A bidirectional layer doubles the LSTM weights (16640 against 8320); two bidirectional layers have 41472, because the second layer reads 64 numbers per word. The embedding table is still most of every model (75505 to 109201 in total).

WHY CHOOSE THIS?

Two switches, one class: all four taggers share every line of code except the two new ideas.

WHY NOT THIS? (TRADEOFFS)

Every extra direction or layer adds a full LSTM's worth of weights and time.

2. Checks: reversing padded sentences, and gradients through it all

DEFINITION — start here, no prior knowledge assumed

Sentences in a batch have different lengths, so shorter ones are **padded** at the end. Reversing a padded sentence must flip only its real words and leave the padding where it is. The gradient check then covers the whole 2-layer bidirectional tagger.

REAL-WORLD SCENARIO

Reading a form backwards: you start at the last line that was actually filled in, not at the blank lines below it.

MATHEMATICS

$$R(x)_t = x_{len-1-t} \quad \text{for } t < len, \quad x_t \text{ otherwise;} \quad R(R(x)) = x$$

$$\text{backward pass: } dX = dX_f + R(\, dX_b\,), \quad \text{where } LSTM_b\text{'s backward pass receives } R(\, d{\leftarrow}H\,)$$

SPECIFICATION

(a) Reversal	Two padded sentences, reversed once and twice.
(b) Gradients	Day 62's tiny data (12 words, 3 tags, embedding 4, hidden 5); 2 layers, bidirectional; all 11 parameter arrays; Day 62's pass rule.
(c) Dependence	Change the LAST word of a sentence; does the FIRST word's prediction move?

Why this test

Reversal is where a bidirectional implementation usually breaks: a flip that moves padding to the front, or a backward pass that forgets to flip the gradient back, gives wrong results without any error message.

IMPLEMENTATION

```
def reverse_padded(A, lens):
    T = A.shape[1]
    t = np.arange(T)[None, :]
    idx = np.where(t < lens[:, None], lens[:, None] - 1 - t, t)
    if A.ndim == 3:
        idx = idx[:, :, None]
    return np.take_along_axis(A, idx, axis=1)
```

IMPLEMENTATION

```
if self.bi:
    dHr = reverse_padded(d_out[:, :, H:], self.lens)
    grads[f"W{l}b"], grads[f"b{l}b"], dXr, _, _ = lstm_backward(dHr, zeros, zeros, caches["b"])
    d_in = d_in + reverse_padded(dXr, self.lens)
```

ACTUAL OUTPUT

```
(a) sentences [[5, 6, 7, 0, 0], [8, 9, 0, 0, 0]] -> reversed [[7, 6, 5, 0, 0], [9, 8, 0, 0, 0]];
    reversed twice equals the original: True
```

ACTUAL OUTPUT

```
(b)  parameter  entries  max relative error  max |difference|   pass
             E       48            5.89e-08          1.34e-11   True
           W0f      180            8.36e-04          1.77e-11   True
           b0f       20            2.67e-06          6.74e-11   True
           W0b      180            6.60e-05          1.80e-11   True
           b0b       20            2.70e-06          2.31e-11   True
           W1f      300            6.52e-05          2.13e-11   True
           b1f       20            4.96e-08          4.65e-11   True
           W1b      300            7.46e-04          1.70e-11   True
           b1b       20            3.67e-07          3.02e-11   True
           Why       30            7.80e-08          2.35e-11   True
            by        3            9.36e-09          1.45e-11   True
every entry passes: True;  <pad>'s embedding gradient exactly zero: True
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
changing the LAST word moves the FIRST word's tag probabilities by: bidirectional 3.91e-04, left
    to right 0.00e+00
Actual output from this lesson's run.
```

What it means

Padding stays in place, two flips restore the original, and every gradient entry passes (the largest relative error, 8.36e-04, is on an entry whose gradient is nearly zero: absolute difference 1.77e-11). Changing the last word moves the first word's prediction in the bidirectional tagger (3.91e-04, small because the tiny network is untrained) and not at all in the left-to-right one (0.00e+00), exactly as designed.

WHY CHOOSE THIS?

Three cheap checks that catch the bugs a bidirectional model typically has.

WHY NOT THIS? (TRADEOFFS)

They check correctness, not usefulness; Section 3 measures that.

3. Training the four taggers

DEFINITION — start here, no prior knowledge assumed

All four taggers trained exactly like Day 67's: three seeds each, test accuracy read at the epoch with the best validation accuracy. A **size control** (a 1-layer left-to-right tagger with hidden 50) has about as many LSTM weights as the 1-layer bidirectional tagger, to separate “reads both ways” from “is bigger”.

REAL-WORLD SCENARIO

Same exam, same study time, four study methods, three students each.

SPECIFICATION

Runs	5 taggers × seeds 0, 1, 2; 25 epochs; lr 4.0; forget bias 1.0.
Reference	Day 67's LSTM tagger: 0.8377; most-frequent-tag baseline 0.8016.

Why this test

The headline question: does reading both ways, or adding a layer, improve tagging?

IMPLEMENTATION

```
rows = CONFIGS[:1] + [("1 layer, left to right, h 50", dict(layers=1, bidirectional=False,
    hidden_dim=50))] + CONFIGS[1:]
for name, cfg in rows:
    accs, preds, epochs, trains = [], [], [], []
    for s in SEEDS:
        net, be, h = fit_cached(data, V, K, seed=s, **cfg)
        accs.append(h["test"][be]); epochs.append(be); trains.append(h["train"][-1])
        preds.append(net.predict(ids_te))
    results[name] = (accs, preds)
```

ACTUAL OUTPUT

```
tagger                          seed 0   seed 1   seed 2     mean   best epochs  train at end
1 layer, left to right          0.8395   0.8370   0.8366   0.8377      13/10/13        0.9561
1 layer, left to right, h 50    0.8392   0.8402   0.8386   0.8393        9/12/9        0.9674
2 layers, left to right         0.8347   0.8347   0.8282   0.8326      18/12/13        0.9375
1 layer, bidirectional          0.8600   0.8556   0.8575   0.8577       17/6/20        0.9862
2 layers, bidirectional         0.8552   0.8540   0.8580   0.8557      18/12/15        0.9828
most-frequent-tag baseline on test: 0.8016; one standard error about 0.0026
Actual output from this lesson's run.
```

What it means

Reading both ways is the big step: 0.8577 against 0.8377, two percentage points, about eight standard errors (0.0026). Every bidirectional seed (0.8600, 0.8556, 0.8575) beats every left-to-right seed. The size control rules out size: the left-to-right tagger with as many weights reaches only 0.8393.

A second layer does not help here. Left to right it is slightly worse (0.8326), bidirectional it is level (0.8557 against 0.8577). The 1-layer left-to-right tagger reproduces Day 67 exactly (0.8395, 0.8370, 0.8366).

For scale: Day 62 found that *at least one* of a left-to-right and a right-to-left tagger was right on about 90% of test words (0.9020 for seed 0). A bidirectional tagger can in principle combine the two; it closes part of that gap, not all of it.

WHY CHOOSE THIS?

Reading both ways gave the biggest single gain of Weeks 9–10, at about twice the cost.

WHY NOT THIS? (TRADEOFFS)

It needs the whole sentence before tagging any word, so it cannot tag text as it is being typed or generated.

3b. Is Day 67's learning rate still right for the bigger taggers?

DEFINITION — start here, no prior knowledge assumed

A setting chosen for one model may not suit a bigger one. Here each bigger tagger is also trained at half the learning rate, seed 0, and compared on **validation** accuracy only.

REAL-WORLD SCENARIO

Before blaming a new recipe, check the oven temperature still suits the larger tray.

IMPLEMENTATION

```
for name, cfg in CONFIGS[1:]:
    _, be, h = fit_cached(data, V, K, seed=0, **cfg)
    _, be2, h2 = fit_cached(data, V, K, seed=0, lr=LR / 2, **cfg)
    print(f"{name:<26}{h['val'][be]:>15.4f}{h2['val'][be2]:>15.4f}")
```

ACTUAL OUTPUT

```
tagger                      val at lr 4.0  val at lr 2.0
2 layers, left to right            0.8282         0.8233
1 layer, bidirectional             0.8547         0.8563
2 layers, bidirectional            0.8431         0.8456
Actual output from this lesson's run.
```

What it means

Halving the learning rate changes validation accuracy by less than the noise (about 0.005 on 6,634 words): 0.8282 vs 0.8233 for two left-to-right layers, 0.8547 vs 0.8563 and 0.8431 vs 0.8456 for the bidirectional ones. The learning rate is not why the second layer fails to help.

WHY CHOOSE THIS?

Rules out the most likely setting-based explanation, cheaply.

WHY NOT THIS? (TRADEOFFS)

One seed and one alternative rate; it does not search the space.

4. Where reading both ways helps

DEFINITION — start here, no prior knowledge assumed

Day 62's word groups (unambiguous, ambiguous, unknown), and the tags whose accuracy changes most from the left-to-right tagger to the bidirectional one.

REAL-WORLD SCENARIO

Checking which kinds of exam question the new study method actually helped with.

SPECIFICATION

Groups	Mean of three seeds per tagger.
Tags	Seed 0 of each 1-layer tagger; the six tags with the largest gain.

Why this test

If looking ahead matters, it should help most where the current word alone is not enough.

IMPLEMENTATION

```
for k in range(K):
    sel = mask_te & (tags_te == k)
    a_u, a_b = (p_uni == tags_te)[sel].mean(), (p_bi == tags_te)[sel].mean()
    stats.append((a_b - a_u, k, int(sel.sum()), a_u, a_b))
```

ACTUAL OUTPUT

```
                            all words  unambiguous    ambiguous      unknown
share of test words            100.0%        37.7%        42.2%        20.2%
most-frequent-tag               0.8016       0.9814       0.8474       0.3705
1 layer, left to right          0.8377       0.9732       0.8749       0.5072
2 layers, left to right         0.8326       0.9697       0.8684       0.5017
1 layer, bidirectional          0.8577       0.9728       0.9083       0.5374
2 layers, bidirectional         0.8557       0.9714       0.9069       0.5329
Actual output from this lesson's run.
```

ACTUAL OUTPUT

```
 true tag  test words  left to right  bidirectional   change
      NUM         376         0.5346         0.6622  +0.1277
    SCONJ         250         0.5960         0.6920  +0.0960
     VERB        1930         0.7762         0.8627  +0.0865
      ADP        1447         0.8576         0.9337  +0.0760
     INTJ          96         0.4896         0.5417  +0.0521
      ADJ        1381         0.6937         0.7415  +0.0478
Actual output from this lesson's run.
```

What it means

The gain is on **ambiguous** words (0.8749 to 0.9083) and **unknown** words (0.5072 to 0.5374); unambiguous words were already right (0.9732 and 0.9728). By tag, verbs gain +0.0865 on 1930 test words, prepositions (ADP) +0.0760, numbers +0.1277: words like “to”, “that” or a verb form whose role is decided by what comes next.

WHY CHOOSE THIS?

The gain is exactly where looking ahead should help.

WHY NOT THIS? (TRADEOFFS)

Group-level numbers are noisier than the total.

5. Which words does each tagger look at?

DEFINITION — start here, no prior knowledge assumed

Day 67's probe in both directions: replace the word k positions before (k < 0) or after (k > 0) a target word with a random word, and measure the total-variation change of the target's tag distribution.

REAL-WORLD SCENARIO

Changing one word in the sentence before or after the one a reader is looking at, and seeing whether their reading of that word changes.

MATHEMATICS

$$TV(p, q) = \tfrac{1}{2} \Sigma_k \, | \, p_k - q_k \, |$$

SPECIFICATION

Edits	Offsets –8, –4, –2, –1, +1, +2, +4, +8; 5 random draws per sentence; the same edits for every tagger.
Models	Three seeds each of the 1-layer left-to-right, 1-layer bidirectional and 2-layer bidirectional taggers.

Why this test

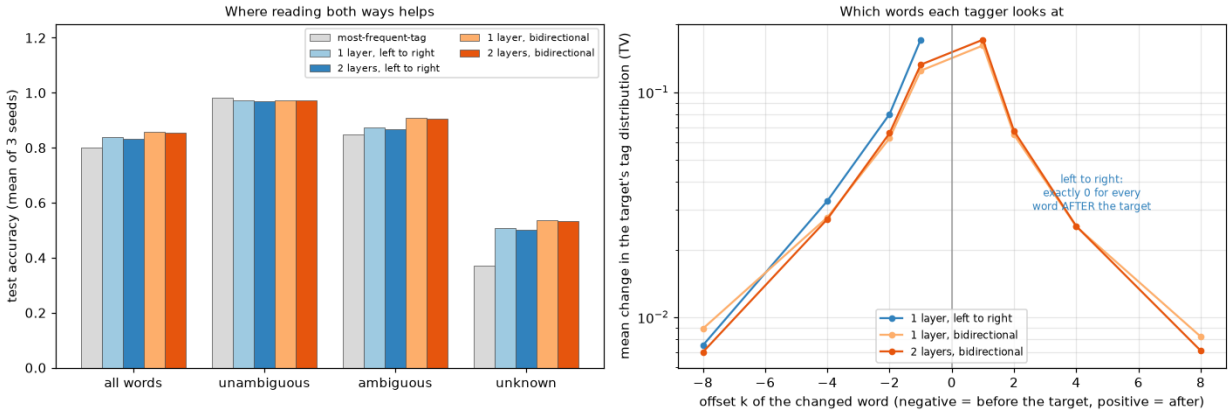
It shows directly whether the bidirectional tagger uses the words after the target, and how much.

IMPLEMENTATION

```
rows = np.where(lens > abs(k))[0]
draws = []
for _ in range(5):
    for r in rows:
        t = rng.randint(max(0, -k), lens[r] - max(0, k))
        draws.append((r, t, rng.randint(2, V)))
draws = np.array(draws)
r_idx, t_idx, new_w = draws[:, 0], draws[:, 1], draws[:, 2]
changed = ids_te[r_idx].copy()
changed[np.arange(len(draws)), t_idx + k] = new_w
keep = changed[np.arange(len(draws)), t_idx + k] != ids_te[r_idx, t_idx + k]
```

ACTUAL OUTPUT

```
 offset k    1 layer, left to right    1 layer, bidirectional   2 layers, bidirectional
       -8                    0.0075                    0.0089                    0.0070
       -4                    0.0329                    0.0278                    0.0273
       -2                    0.0800                    0.0625                    0.0658
       -1                    0.1713                    0.1247                    0.1326
       +1                    0.0000                    0.1614                    0.1712
       +2                    0.0000                    0.0647                    0.0672
       +4                    0.0000                    0.0253                    0.0254
       +8                    0.0000                    0.0082                    0.0071
Actual output from this lesson's run.
```



Left: accuracy by word group. Right: how much changing the word at offset k moves the target's tag distribution (log scale; the left-to-right tagger's values after the target are exactly 0 and cannot be drawn on it).

What it means

The left-to-right tagger is exactly blind to every later word (0.0000). The bidirectional tagger uses the next word (0.1614) about as much as the previous one (0.1247), and leans a little less on the previous word than the left-to-right tagger does (0.1713): it now has two neighbours to go on. Influence fades with distance on both sides about equally; tagging stays a short-range task, just in both directions.

WHY CHOOSE THIS?

Shows the mechanism behind the accuracy gain, not just the gain.

WHY NOT THIS? (TRADEOFFS)

One random edit at a time; it does not measure interactions between several words.

6. What each costs

DEFINITION — start here, no prior knowledge assumed

Training seconds per epoch, median over every epoch of the three seeds, matrix library on one thread.

REAL-WORLD SCENARIO

Two proofreaders cost twice as much as one; two rounds of proofreading too.

IMPLEMENTATION

```
secs = np.median(np.concatenate([fit_cached(data, V, K, seed=s, **cfg)[2]["seconds"] for s in
    SEEDS]))
base = secs if base is None else base
print(f"{name:<26}{secs:>17.2f}{secs / base:>27.2f}")
```

ACTUAL OUTPUT

```
tagger                      seconds / epoch  vs 1 layer, left to right
1 layer, left to right                 0.50                       1.00
2 layers, left to right                0.90                       1.80
1 layer, bidirectional                 0.98                       1.97
2 layers, bidirectional                2.04                       4.09
Actual output from this lesson's run.
```

What it means

A second direction or a second layer each roughly doubles the time (1.97 and 1.80); both together cost 4.09 times as much. The bidirectional layer is the one worth paying for here.

WHY CHOOSE THIS?

Measured on the real workload.

WHY NOT THIS? (TRADEOFFS)

Timings depend on the machine and the sizes.

Interview preparation — technical questions

Q: How do you implement a bidirectional RNN with padded batches?

A: Reverse each sequence's real part only (padding stays at the end), run a second RNN on it, flip its outputs back and concatenate with the forward outputs. In the backward pass, flip the gradient the same way.

Q: What is the input size of the second layer of a bidirectional stack?

A: Twice the hidden size: it reads both directions' states joined.

Q: How would you check that a bidirectional model really uses future context?

A: Change a word after the target and measure whether the prediction moves; for a left-to-right model it must not move at all.

Q: Your 2-layer model is no better than 1 layer. What do you check?

A: The learning rate (here halving it changed nothing beyond noise), overfitting (training accuracy near 0.98), and whether the task needs more abstraction at all.

Q: How much more does a bidirectional layer cost?

A: About twice the weights and twice the time of one direction.

Interview preparation — theoretical questions

Q: When can you not use a bidirectional RNN?

A: When outputs are needed before the input is complete: real-time speech, live captioning, or generating text word by word (a language model or a decoder).

Q: Why might stacking not help?

A: Small data: more layers mean more weights to overfit, and part-of-speech tagging may not need a second level of abstraction. Deep RNNs help more on large datasets and harder tasks such as translation or speech.

Q: Why did the bidirectional tagger help ambiguous words most?

A: Their tag depends on context the current word does not give, and often on what follows (“to” + verb vs “to” + noun).

Q: Bidirectional or bigger?

A: Here a same-size left-to-right tagger gained 0.2 points, the bidirectional one 2 points: direction, not size.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Letting the tagger see the words after each word was worth two points (0.8377 to 0.8577), far more than any change of cell on Day 67, and not because the model is bigger. The gain is on ambiguous and unknown words, and the probe shows the bidirectional tagger using the next word as much as the previous one. A second layer added time but no accuracy on this small dataset, at either learning rate tried.

- Bidirectional: two RNNs, one per direction, joined at every position; reverse only the real part of padded sequences.
- Test accuracy: 1 layer left to right 0.8377, bidirectional 0.8577; 2 layers 0.8326 and 0.8557.
- Size control (left to right, hidden 50): 0.8393; the gain is the direction, not the weights.
- Biggest gains: ambiguous words, verbs, prepositions, numbers.
- Left to right is exactly blind to later words; bidirectional weighs the next word like the previous one.
- Each extra direction or layer about doubles the training time.

Suggested watch order, if starting from zero

- 1. Bidirectional RNN | Deep Learning Tutorial 38 (Tensorflow, Keras & Python) (codebasics) — why and how a bidirectional RNN reads both ways.
- 2. Deep RNNs | Stacked RNNs | Stacked LSTMs | Stacked GRUs (CampusX) — stacking recurrent layers, in depth.

Day 69 preview

- **Sequence to sequence, the week's build.** A bidirectional layer needs the whole input first, so it suits an *encoder* that reads an input sequence; a *decoder* that writes an output one token at a time must read left to right. Day 69 assembles an encoder-decoder from this week's exercise scripts and trains it to copy and to reverse sequences.
- Today's exercise script, `day68_exercise.py`, isolates a bidirectional LSTM layer: it solves “output the next token”, which a left-to-right layer cannot.